

Prompt Engineering

From Clear Instructions to Reusable LangChain Pipelines

A conceptual guide for professionals building reliable, structured LLM-powered workflows — no training required.



What We'll Cover

01

Foundations

What prompt engineering is, how it differs from training, and how to structure an effective prompt.

03

Reusable Templates

PromptTemplate, ChatPromptTemplate, variables, and building pipelines in LangChain.

02

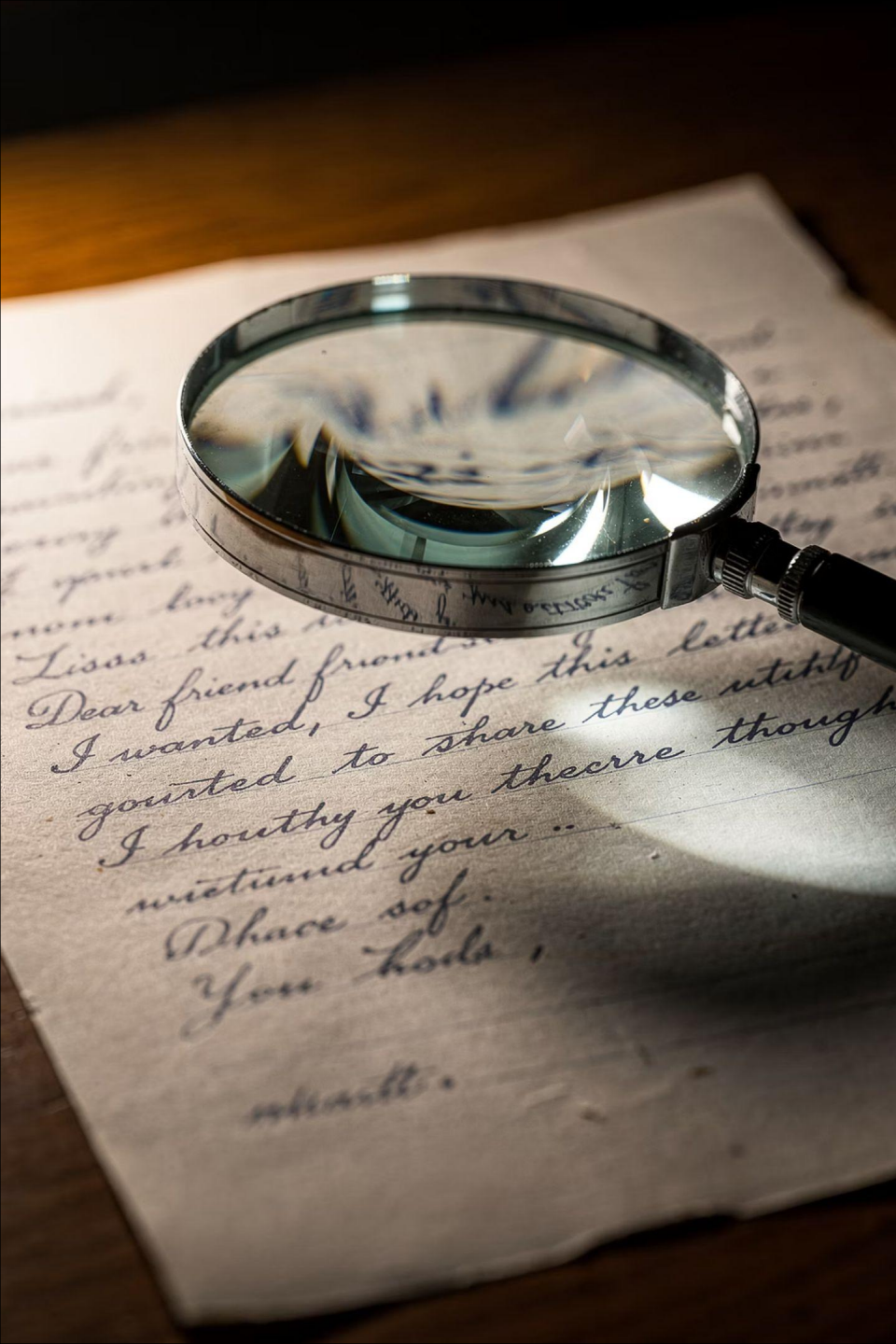
Prompt Techniques

Zero-shot vs. few-shot, roles, delimiters, output formatting, and common mistakes to avoid.

04

Output & Limits

Parsing, validation, pipeline composition, and when prompting alone isn't enough.



CHAPTER 1

Foundations of Prompt Engineering

Understanding what prompting is, how it works, and why good instructions are the single most powerful lever you have when working with LLMs.

What Is Prompt Engineering?

Definition

Prompt engineering is the practice of crafting the text input you send to a language model so it produces the output you actually need.

A prompt is your specification — the model reads it and generates a response that satisfies the described task. Better specifications produce better, more consistent results.

Why It Matters

- LLMs are general-purpose: they do many things, but they don't know *your* task without being told.
- The same model can summarize, classify, extract, or translate — the prompt is what directs it.
- Prompts are editable in seconds; model retraining takes weeks.

Prompting vs. Training vs. Fine-Tuning

These are three distinct approaches to influencing model behavior. Knowing the difference prevents overengineering and misplaced effort.

Prompting

Instructions at inference time. No data, no GPU, no waiting. Changes take effect immediately. Works within the model's existing knowledge.

Fine-Tuning

Retrains the model on your labeled examples. Teaches new style, domain vocabulary, or behavior — but requires data, compute, and time.

Pre-Training

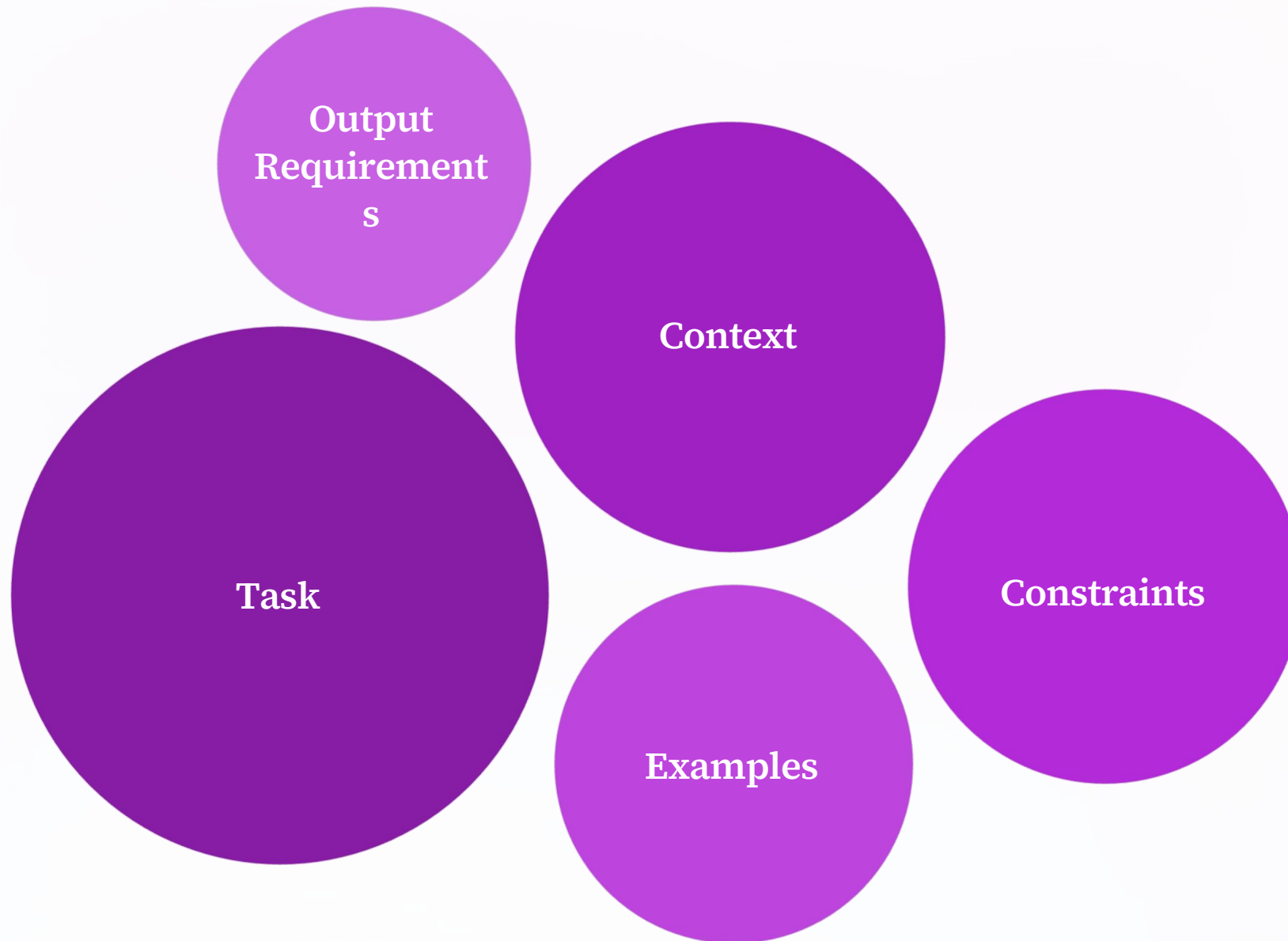
Training from scratch on massive text corpora. Determines the model's base knowledge and language understanding. Done by model developers, not application teams.



Start with prompting. Only consider fine-tuning when prompting consistently falls short on a specific, well-defined capability.

Anatomy of an Effective Prompt

Most well-performing prompts share five core components. Not every prompt needs all five, but understanding each helps you diagnose what's missing when a prompt underperforms.



Writing Clear Instructions

Use Specific Action Verbs

Vague verbs like "deal with" or "handle" leave too much open to interpretation. Precise verbs anchor the model to a single operation.

Weak	Stronger
Handle this message	Classify, summarize, and recommend
Write something about X	Write a 3-sentence explanation of X for a non-technical reader
Make it better	Rewrite for a formal tone, under 100 words

Scope and Audience

Scope tells the model *how much* to cover. Audience tells it *how to* communicate. Both prevent the model from defaulting to a generic, unfocused response.

- **Scope:** "Summarize in two bullet points" vs. "Summarize comprehensively"
- **Audience:** "Explain to a first-year support agent" vs. "Explain to a senior engineer"

Context: Useful vs. Unnecessary

What Is Context?

Context is background information the model needs to complete the task correctly. Without it, the model fills gaps with assumptions that may not match your situation.

Useful context example:

Customer message: "My order hasn't arrived after 10 days."
Policy: Orders ship within 3 business days. Delays beyond 7 days qualify for expedited re-shipment.

The Context Test

Before adding background, ask: *Would a skilled analyst need this to do the task?*

- **Include:** Source documents, policies, relevant history, field definitions
- **Exclude:** Company founding history, tangential facts, repeating the task description as context

Excess context dilutes the signal. The model attends to everything in the prompt — irrelevant content competes with relevant content.

Constraints: Shaping the Response

Constraints are explicit limits on what the model should produce. They reduce variance and make outputs consistently usable downstream.

Length

Set a word count, sentence count, or item limit. "Respond in at most three sentences" produces shorter, denser output than "be concise" — which is unspecified.

Tone

Specify register: formal, empathetic, direct, technical. Without guidance, tone defaults to the model's general training distribution — not your brand voice.

Scope

Tell the model what to stay out of. "Do not recommend third-party products" or "Only use information from the provided policy document."

Permitted Evidence

Restrict the model to supplied source material when accuracy matters. "Base your answer only on the text below" reduces the risk of hallucinated details.

Delimiters: Separating Instructions from Content

What Delimiters Do

A delimiter is a consistent visual marker — triple quotes `"""`, XML-like tags `<message>`, or dashes — that separates the instruction section from the source material the model should process.

Classify the following customer message.

`<message>`

My invoice is wrong. I was charged twice for the same item last week.

`</message>`

Return: category, priority, summary.

What Delimiters Don't Do

Delimiters improve **structural clarity** — the model reads the prompt in sections rather than as one undifferentiated block. They do **not** provide a security boundary.

- A user who submits content inside the delimited section can still craft adversarial instructions within it.
- Preventing prompt injection requires application-level controls, not punctuation alone.



Delimiters = clarity tool, not a security guarantee.

Roles and Personas

How Roles Work

Assigning a role shifts the **tone, perspective, and framing** of the model's response. Roles appear as a system message in chat-based APIs.

System: You are a professional customer-support specialist. Respond clearly, empathetically, and within the scope of company policy.

This prompts the model to adopt a consistent register across all turns of a conversation.

What Roles Cannot Do

Assigning a persona does **not** grant the model domain expertise it wasn't trained on. Telling a model it is "a licensed physician" does not make its medical outputs clinically reliable.

- Use roles to guide **communication style**
- Use roles to maintain **consistent voice**
- Do **not** rely on roles as a substitute for grounding in authoritative source material

Specifying Output Format

Telling the model exactly what format to return dramatically increases downstream usability. Unstructured prose is hard to parse; specified formats integrate directly into applications.

Prose

"Write a two-paragraph summary in plain English." Best for human-readable responses where flow matters more than machine parsing.

Tables

"Return a markdown table with columns: Issue, Category, Priority."
Useful for comparison, review, or display in UIs that render markdown.

Structured Fields

"Return JSON with keys: category, priority, summary, recommended_action." Ideal for downstream parsing, storage, or routing logic in an application.

CHAPTER 2

Prompting Techniques

Zero-shot and few-shot prompting, example quality, evaluation, and the most common mistakes that undermine otherwise solid prompts.



Zero-Shot Prompting

Definition

Zero-shot prompting means giving the model a task description and no examples. The model relies entirely on its pre-trained knowledge to understand what's expected.

Classify the following customer message
as: Billing, Shipping, or Technical.

Message: "I can't log in to my account
after changing my password."

Category:

Illustrative Output

Category: Technical

The model correctly infers the classification schema from the label list alone. Zero-shot works well when:

- The task is straightforward and common in training data
- The output format is simple
- You have no labeled examples ready

When zero-shot produces inconsistent results, few-shot is the next step.

Few-Shot Prompting

Definition & Structure

Few-shot prompting includes one or more input–output examples before the actual task. Examples demonstrate the expected format, reasoning style, and level of detail.

Classify each message as Billing,
Shipping, or Technical.

Message: "My credit card was charged
twice this month."
Category: Billing

Message: "My package shows delivered
but I haven't received it."
Category: Shipping

Message: "The app crashes when I open
the settings screen."
Category:

Illustrative Output

Category: Technical

The examples teach the model the *pattern* of classification without any labeled training data.

When to Choose Few-Shot

- Output format is complex or unusual
- Zero-shot is inconsistent
- You need a specific reasoning style or level of detail

Zero-Shot vs. Few-Shot: Comparing the Tradeoffs

Neither approach is universally better. The right choice depends on task complexity, example availability, and token budget.

Dimension	Zero-Shot	Few-Shot
Setup effort	Minimal — no examples needed	Requires curated examples
Token usage	Lower	Higher (examples consume context)
Output consistency	Variable on novel tasks	More consistent with good examples
Format control	Depends on task description	Strong — examples show the format
Risk of bias	Lower — no example patterns	Higher — poor examples mislead

 Start zero-shot. Add examples incrementally if the output format is inconsistent or the reasoning pattern is off.

Example Quality in Few-Shot Prompting

What Makes a Good Example

- **Representative:** Covers the range of inputs the model will actually see, not just easy cases.
- **Consistent:** All examples follow the same output format exactly — inconsistent formatting confuses the model.
- **Correct:** Every example must be accurate. The model treats examples as ground truth.

Risks of Poor Examples

- **Misleading patterns:** If all examples are billing issues, the model may over-predict "Billing" for ambiguous messages.
- **Format drift:** Mixing JSON and prose across examples produces unpredictable output.
- **Outdated facts:** Examples containing stale information get treated as valid reference by the model.



One bad example can degrade an otherwise excellent prompt.

Comparing a Weak Prompt to an Improved Prompt

✗ Weak Prompt

Read this message and respond appropriately.

"My order is late."

Problems identified:

- "Respond appropriately" — task is undefined
- No output format specified
- No context (policy, tone, audience)
- No constraints on length or scope

✓ Improved Prompt

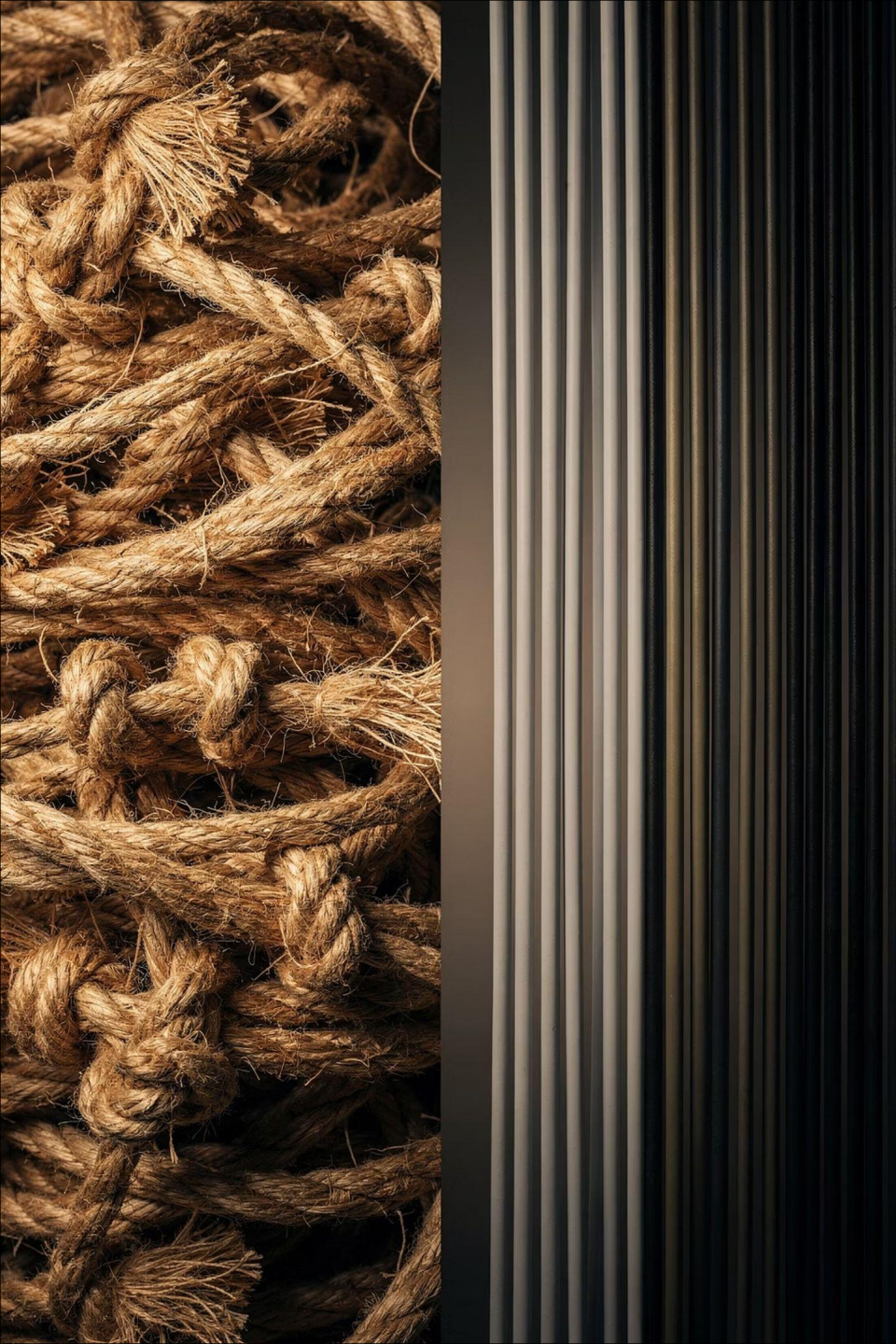
You are a customer-support specialist. Classify the message below as Billing, Shipping, or Technical. Then write a one-sentence empathetic reply based only on the provided policy.

Policy: Orders ship in 3 days. Delays over 7 days qualify for re-shipment.

<message>My order is late.</message>

Return JSON: {category, reply}

What changed: role added, task clarified with action verbs, policy context provided, delimiter added, output format specified.



Common Prompt Mistakes



Vague Goals

"Summarize this" — summarize *for whom*, *how long*, and *which aspects*?



Conflicting Instructions

"Be concise but comprehensive" — the model can't optimize both simultaneously without guidance on which takes precedence.



Missing or Excessive Context

Too little: the model fills gaps with assumptions. Too much irrelevant detail: important signals get buried.



Unspecified Output Format

Without a format, each run may return prose, bullets, JSON, or something else — making downstream processing brittle.

Evaluating Prompt Quality

A prompt's quality is measured by whether its outputs meet task-specific criteria — not by how it looks or how long it is. Evaluation is iterative: test, identify failure modes, revise.

1

Correctness

Does the output accurately reflect the source material or task requirements? Check facts, classifications, and extracted values against ground truth.

2

Completeness

Are all required output fields or sections present? A JSON response missing a key is incomplete regardless of the quality of other fields.

3

Format Compliance

Does the output match the specified structure — JSON, table, prose length? Inconsistent format breaks downstream parsing.

4

Source Fidelity

When you've restricted the model to supplied evidence, does the output stay within it? Hallucinated references violate this criterion.



CHAPTER 3

Reusable Templates & LangChain Pipelines

Moving from one-off prompts to structured, parameterized templates that scale across use cases and integrate cleanly into application code.

Reusable Prompt Templates

What Is a Prompt Template?

A **prompt template** is a prompt with placeholders for the parts that change across requests. The stable instructions — task, role, constraints, format — are written once. Only the variable inputs change at runtime.

```
Classify the following customer message
as Billing, Shipping, or Technical.
Reply in a {response_format} using a
{tone} tone.
```

```
Policy context:
{policy_context}
```

```
Message:
{customer_message}
```

Why Templates Matter

- **Consistency:** Every request uses the same tested instruction structure.
- **Maintainability:** Update the instruction in one place, not across every call.
- **Reusability:** The same template serves multiple customers, topics, or use cases by substituting different variables.
- **Testability:** You can evaluate the template independently from any specific input.

PromptTemplate vs. ChatPromptTemplate

LangChain provides two core template classes that correspond to the two main interaction patterns for language models.

Feature	PromptTemplate	ChatPromptTemplate
Input format	Single string with placeholders	List of message role/content pairs
Output format	Single formatted string	List of BaseMessage objects
Roles supported	No explicit roles	System, human, AI roles
Best for	Completion-style or simple prompts	Chat models like GPT-4o, Claude
Constructed from	PromptTemplate.from_template()	ChatPromptTemplate.from_messages()

For modern chat models, ChatPromptTemplate is the standard choice. It lets you set a persistent system instruction separately from user messages.

Variables and Parameterized Prompts

Placeholders as Variables

Variables are written as `{variable_name}` inside the template string. At runtime, LangChain substitutes each placeholder with the actual value passed in.

Common variables for a support pipeline:

- `{customer_message}` — the incoming support request
- `{policy_context}` — the relevant policy section
- `{response_format}` — prose or JSON
- `{tone}` — empathetic, formal, concise

Good Variable Design

- Name variables after what they *contain*, not how they're used
- Keep each variable responsible for one piece of information
- Document expected types and constraints alongside the template

 Variables that cover multiple concerns (e.g., a single `{context}` for both policy and customer history) become hard to manage. Separate them.

From Template + Variables → Populated Prompt

Template
Prompt with
placeholders like
{customer_message}

Populated Prompt
Filled prompt sent to
model for classification



Runtime Variables
Provide values such as
customer text and policy

Template (static)

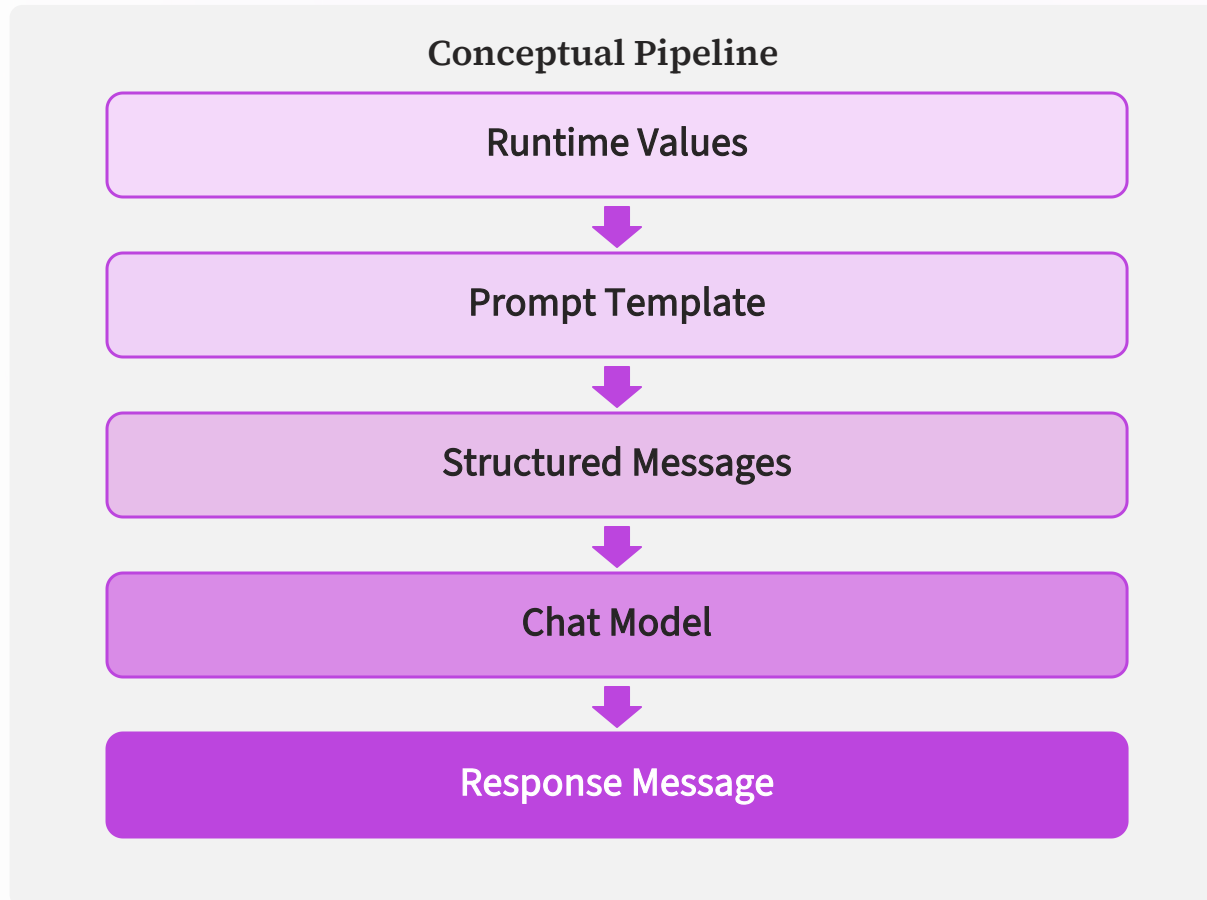
```
Classify {customer_message} as  
Billing, Shipping, or Technical.  
Policy: {policy_context}  
Return JSON.
```

Populated Prompt (runtime)

```
Classify "My invoice is incorrect"  
as Billing, Shipping, or Technical.  
Policy: Billing disputes must be  
submitted within 30 days.  
Return JSON.
```

Connecting a Prompt to a Chat Model

A reusable pipeline passes runtime values through a prompt template, converts the result into messages, and sends those messages to a chat model.



Conceptual Flow

1. Application supplies the runtime values
2. The template fills its variable placeholders
3. The populated template produces structured messages
4. The chat model generates a response message
5. Output handling prepares the result for the application

Each stage has a clear input and output. The concept applies across different model providers.

Basic Output Handling

The model's raw response is a string (or `AIMessage`). Your application often needs a structured value. Output handling bridges that gap.

1

Text Extraction

Pull the text content from the response object using `.content` on an `AIMessage`.

2

Parsing

Convert the string to the expected type — parse JSON, split CSV, or pattern-match with a regex.

3

Schema Validation

Check that required keys are present and values are the right type. Tools like Pydantic make this explicit and testable.

4

Handling Invalid Output

If parsing or validation fails, decide: retry the prompt, return a default, or surface an error — don't silently pass bad data.

Three Layers of Output Structure

There's an important distinction between how you *ask* for structure, how the API *enforces* it, and how your application *validates* it. Conflating these layers leads to brittle pipelines.

Prompt-Requested Format

"Return JSON with keys: category, priority, summary." The model *tries* to comply but can produce malformed JSON, prose explanations, or extra keys. Not guaranteed.

Provider Structured Output

Some providers (e.g., OpenAI's `response_format` or tool-calling APIs) constrain the model's generation to a valid schema. More reliable, but model- and provider-specific.

Application-Side Validation

Your code validates the parsed response against a schema (e.g., Pydantic). This is the only layer fully under your control, regardless of what the model returns.

Pipeline Composition: End-to-End Flow



Each stage has a single responsibility. When a pipeline misbehaves, this separation makes it straightforward to isolate whether the issue is in the prompt, the model's response, the parser, or the validation layer — rather than debugging the entire system at once.

The Support-Message Example: Field by Field

Throughout this course we've used a customer-support routing task as a shared example. Here's exactly what each output field means and how the model is expected to derive it.

Field	How Derived	Important Note
category	Classified from the message text against a fixed label list in the prompt	Label list must be defined in the prompt; don't leave it open-ended
priority	Inferred from urgency signals in the message and any policy rules provided	Model-assigned priority is a suggestion — human review is advisable for high-stakes routing
summary	A concise restatement of the customer's core issue in neutral language	Should reflect the message content, not invent details
recommended_action	A next step grounded in the provided policy context	Only as reliable as the policy supplied; absent policy → generic advice

Illustrative Support Pipeline Output

Input Message

"I've been waiting 12 days for my order. No one has responded to my previous email. This is urgent."

Policy Provided

Orders delayed beyond 7 days qualify for expedited re-shipment. Escalate cases with prior unanswered contact.

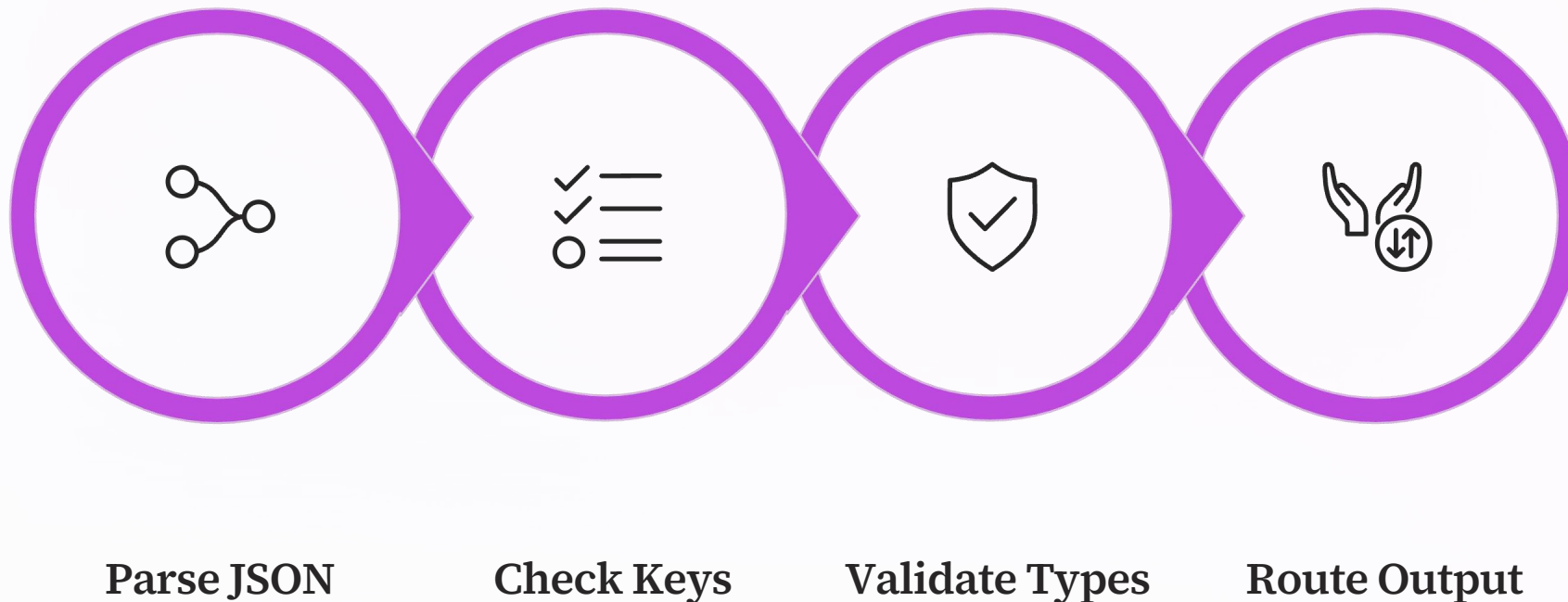
Illustrative Model Output

```
{  
  "category": "Shipping",  
  "priority": "High",  
  "summary": "Customer reports a 12-day  
    delay with no prior response to  
    their support email.",  
  "recommended_action": "Initiate  
    expedited re-shipment and escalate  
    due to previous unanswered contact."  
}
```



This is an illustrative example. Actual output depends on the model version and exact prompt.

Output Validation Flow



LangChain's `PydanticOutputParser` and `JsonOutputParser` automate much of this process. They raise structured exceptions on failure, which you can catch and handle — retry the model, substitute a default, or propagate the error to a monitoring system.

When Prompting Isn't Enough

The Core Limitation

A language model can only work with what's in its context window: its training knowledge plus whatever you supply in the prompt. When the required information is **absent from both**, the model cannot produce a grounded answer.

Common failure modes:

- Asking about events after the model's training cutoff
- Queries requiring specific internal documents or databases
- Tasks needing up-to-date pricing, inventory, or case records

Introducing Retrieval

Retrieval-Augmented Generation (RAG) addresses this by fetching relevant documents at query time and inserting them into the prompt as context — bridging the gap between what the model was trained on and what the user actually needs.



RAG is the natural next step after mastering prompt templates and pipelines. LangChain provides first-class support for retrieval chains.

Recap: Key Concepts at a Glance

01

Prompt Anatomy

Every effective prompt has a task, context, constraints, optional examples, and a specified output format. Missing components cause predictable failures.

02

Techniques

Zero-shot for simple tasks; few-shot when format consistency matters. Role and delimiter clarity improve structural reliability.

03

Templates

Separate stable instructions from changing inputs using `ChatPromptTemplate`. Parameterize with named variables. Test templates independently.

04

Pipelines

Chain template → model → parser → validator using LCEL. Each stage has one responsibility; isolate failures by stage.

05

Limits

Prompts can't supply information the model doesn't have. When grounding is required, retrieval (RAG) is the next layer to add.

Troubleshooting Summary

Symptom	Likely Cause	Fix
Output format varies across runs	No output format specified	Add explicit format instruction and example
Wrong classification label	Ambiguous or missing label list	Enumerate all valid labels in the prompt
Model ignores constraints	Conflicting instructions or buried constraints	Move constraints to system message; remove conflicts
Hallucinated details	No source grounding; model fills gaps	Add policy/source text; restrict to supplied evidence
JSON parse error in app	Model returned prose or malformed JSON	Use JsonOutputParser; add retry on failure
Few-shot examples biasing output	Examples not representative of full input range	Balance and diversify example set
Model lacks required knowledge	Information absent from context and training data	Supply via context; consider RAG for dynamic data

References

LangChain Documentation

- [Prompt Templates — LangChain Concepts](#)
- [LangChain Expression Language \(LCEL\)](#)
- [Output Parsers](#)
- [Structured Output](#)

Foundational Papers

- Brown et al. (2020). *Language Models are Few-Shot Learners*. (GPT-3 paper; introduced few-shot prompting)
- Wei et al. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*.

Practical Guides

- [OpenAI Prompt Engineering Guide](#)
- [Anthropic Prompt Engineering Overview](#)
- [Prompt Engineering Guide \(DAIR.AI\)](#)

Output Validation

- [Pydantic Documentation](#)



Verify LangChain links against the current `python.langchain.com` docs — the library evolves rapidly and some module paths change between minor versions.